

PROJECT TITLE: KairosSNNI – Development of an SLO-Aware Orchestrator for Multi-Tenant Secure Inference

TARGET BACKEND: SHARK / FSS-based Secure Computation

HARDWARE TARGET:

P-core Edge Server, Shared Memory M , 1TB Project/Scratch Space

1. PROJECT OBJECTIVE

The developer shall implement the **Secure Inference Service Engine (SISE)**, a multi-tenant orchestration layer designed to manage the unique spatio-temporal resource coupling of Secure Neural Network Inference (SNNI). The system must handle high-volume ephemeral cryptographic states (e.g., 35GB VGG-16 material) and mitigate the "Hostage Core" effect caused by resource-constrained clients while maximizing the global utility metric Ψ

2. CORE DATA STRUCTURES

The developer must implement the following objects within the SISE namespace:

- **Task Profile (τ_i):** Static registry.
 - *Parameters:*
Model n

, Protocol $\mathcal{P}$

, Batch Size B_i

, phase-specific WCET (e_{pre}, e_{on}), memory footprints (m_{pre}, m_{on}), and

a recommended alternative model n_{alt} .
- **Request Object ($R_{i,k}$):** Dynamic instance.
 - *Parameters:*
Arrival time (t_{arr}), Due Date ($D_{i,k}$), **Capability Vector** (Cap_c), **Priority Penalty** ($\phi_{i,k}$),
and a verified **Slowness Factor** (θ_c).

- **Capability Vector (Cap_c):** Client hardware self-report.
 - *Fields:* device_class, cpu_freq, available_ram, network_type.

3. COMPONENT IMPLEMENTATION & LOGIC

3.1 Admission Control & Negotiation State Machine (The Multi-Tiered Gatekeeper)

This module is responsible for the **Sequential Validation Process**. As the entry point gateway, this module must implement:

- **Multi-Tiered SLO Filtering:**
 1. **Practicality Filter:**
Reject requests if the requested relative due date ($D_{i,k}$) is shorter than the minimum possible execution time.
Logic: If $D_{i,k} < e_{on}$, notify client and reject.
 2. **Static SLO-factor based Rejection:**
Reject requests that lack sufficient timing margin.
Logic: If $D_{i,k} < (\text{Configurable SLO-factor}) \times e_{on}$, reject the request as "highly likely to violate."
 3. **Dynamic SLO based Rejection (VFT Filter):**
The module must estimate the **Virtual Finish Time (VFT)** by querying the **Telemetry Engine**.
Logic: Calculate VFT using: (Current Global Queue state) + (Core availability) + (Residual time of running jobs). If $VFT > D_{i,k}$, reject the request to prevent "wasting" resources on guaranteed failures.

3.2 Asymmetry Probing & Negotiation

- **Initial Handshake Probe:** Measure **Time-to-First-Reply (TTFR)** during the initial mask distribution handshake.
- **Asymmetry Identification:** Verify Cap_c via measured TTFR to set the initial Slowness Factor (θ_c).
- **Negotiation State Machine:** If θ_c is above threshold, asynchronously propose n_{alt} to the client.
- **Penalty Logic:** If the client refuses the recommendation, set **Priority Penalty** $\phi_{i,k} = 1$

Verification: Compare actual TTFR against Cap_c baseline. If $TTFR_{obs} > 1.2 \times TTFR_{base}$, update the session's θ_c

- **Negotiation Loop:** If θ_c exceeds the threshold, trigger an asynchronous proposal to the client offering n_{alt} . (e.g., shift from $n = \text{AlexNet}$ to $n_{alt} = \text{M2}$)
 - If the client refuses, set **Priority Penalty** $\phi_{i,k} = 1$ (demoting the request in the priority queue).

3.2 Telemetry Engine & Closed-Loop Feedback

- **Dynamic Profiler:** Maintain a thread-safe Profiling Table (Π).
- **The Audit Hook:** Integrate a measurement hook in the SNNI engine to capture the **ground-truth latency of the first ReLU layer**.
- **EWMA Learning:** Update the global database for device classes using an **Exponential Weighted Moving Average**: $\theta_{new} = (1 - \alpha)\theta_{old} + \alpha(\theta_{observed})$
 - *Note:* α must be a configurable smoothing factor (0.1–0.8).

3.3 Request Scheduler (Online & Offline)

- **Offline Scheduler (Replenisher):** Background process managing the 1TB project space. Monitor buffer levels and trigger J_{pre} jobs on a dedicated core quota to ensure "Just-in-Time" file readiness.
- **Online Scheduler (The Brain):** Implement **Simulated Annealing (SA)** and **Deep Q-Network (DQN)** agents.
 - The scheduler must optimize for the **Global Utility Metric (Ψ)**:

$$\text{Maximize } \Psi = \frac{\text{Total Successful SLOs}}{\text{Aggregate Turnaround Time}}$$

- **Malleable Allocation:** Determine the optimal core assignment ($1 \dots P_{max}$) for each J_{on} at the point of dispatch.

3.4 Deterministic Dispatcher

- **Triple Check:** Perform a final verification of physical core IDs, memory capacity, and J_{pre} file presence.
- **HoL Mitigation:** Implement a non-blocking bypass logic to skip "not-ready" requests in the priority queue.
- **Strict Core Pinning:** Enforce physical process binding to specific core IDs and ports to ensure performance isolation between tenants.

4. OPERATIONAL CONSTRAINTS (THE "HARD" RULES)

The developer must ensure the code strictly adheres to these constraints:

1. **Precedence:**

J_{on} cannot start until its specific, single-use J_{pre} file is loaded into RAM.

2. **Non-Preemption:** Once an inference begins, it must run to completion on its assigned cores.
 3. **Global Resource Safety:** Total active cores $\leq P$ and Total Memory (Active + Proactive Buffer) $\leq M$
 4. **Hostage Core Management:** J_{on} duration must be calculated as $\theta_c \cdot e_{on}$
-

5. Scheduling Methodologies

The developer must implement a "Plug-and-Play" interface to switch between these algorithms:

5.1 Baselines (Control Group)

- **First-Come-First-Served (FCFS):** Execute in order of arrival.
- **Shortest Processing Time (SPT) with Aging:** Run the fastest jobs first to maximize throughput.

5.2 SLO-Aware Heuristics

- **Earliest Deadline First (EDF):** Prioritize jobs with the nearest absolute deadline.
- **Least Slack Time (LST):** Calculate "Slack" (Time remaining until deadline - Time required to run). Prioritize jobs with the lowest slack.

5.3 Computational Intelligence

- **Simulated Annealing (SA):** Periodically re-orders the queue to find a sequence that minimizes the total number of missed deadlines.
- **Deep Q-Network (DQN):** Use a pre-trained or online Reinforcement Learning agent to select the next job based on queue state and hardware load.
- **Add a few more related techniques if possible.**

Profiling & Monitoring

- **Online Profiler:** The system must maintain a running "Weight" (Moving Average + Standard Deviation) of how long each model/batch takes to run.
- **Initial Profile:** The system starts with a profile.cfg containing baseline execution times.
- **System Stats:** Continuous logging of CPU/RAM utilization and "Goodput" (successful inferences per second).

6. Provided Base Materials

6.1 Use of Existing Repository:

The Client has provided a private GitHub repository as the foundation for this task. This repository already contains:

1. **Baseline Scheduling Techniques:** (e.g., FCFS, SJF-Aging).
2. **Protocol Backend:** Logic for file preprocessing and bi-phased execution.

6.2 Extension Responsibility:

The Developer's role is to **extend** this existing codebase by integrating the new KairosSNNI components (Admission Control state machine, Dynamic Profiler, SA/DQN Scheduler, and ReLU Audit hooks) into the current architecture without breaking the established protocol functionality.

7. Confidentiality and Intellectual Property Rights

7.1 Absolute Confidentiality:

This project, including its mathematical formulation, architectural design, and experimental results, is strictly confidential. The Developer agrees that no part of the logic, algorithms (specifically the Simulated Annealing and DQN implementations), or system telemetry shall be shared, disclosed, or published to any third party without prior written consent.

7.2 Intellectual Property Ownership:

All work products created under this contract—including source code, profiling data, and documentation—are the exclusive intellectual property of the Client. The Developer waives all rights to the code and shall not reuse the specific "KairosSNNI" logic for other projects or public repositories.

7.3 Prohibition of Public Hosting:

The Developer is strictly prohibited from uploading any portion of the source code or technical documentation to public repositories (e.g., public GitHub, GitLab). All version control must be conducted within the private repository provided by the Client.

8. Performance & ACCEPTANCE CRITERIA

- **Robustness:** The scheduler must handle process failures without crashing. If a SHARK instance fails, the port and cores must be recovered.
- The system must maintain zero Out-of-Memory (OOM) crashes under high-load VGG-16 bursts.
- KairosSNNI must demonstrate a quantifiable improvement in Ψ (Success vs. Latency) compared to standard FCFS and SJF-Aging schedulers.
- Audit logs must verify successful core-pinning and dynamic adjustment of θ_c based on execution ground truth.
- **Concurrency:** Must demonstrate multiple *different* models running at the same time on different ports.

- **SLO Attainment:** In testing (Poisson arrival rate), the SLO-aware algorithms (LST/EDF) must show a higher "Attainment Ratio" than FCFS.
- **Logging:** A comprehensive CSV log (arrival_time, start_time, finish_time, SLO_met_bool, model_type, exit_code, SLO violation %age in 5ms, individual layer's execution times).
- Comparison of Turnaround time for fast clients and a weak client together in the system.

9. Recommended Log Format (CSV)

I recommend your outputs a **Per-Input Trace** in a CSV format. This allows you to generate any graph later in Python/Matplotlib.

Column Header	Description
request_id	Unique (i, k, j) identifier.
model_type	AlexNet, M1, VGG16, etc.
SLO attainment	1 if on time, 0 if late.
t_arr	Arrival timestamp.
t_start_pre	Start of preprocessing.
t_fin_pre	End of preprocessing.
t_start_on	Start of inference.
t_fin_on	Completion timestamp.
theta_est	Slowness factor from handshake.
theta_act	Actual slowness from ReLU hook.
penalty	Was ϕ applied?
cores_assigned	Number of cores ($1 \dots P_{max}$) given by scheduler.
mem_peak	Max RAM used during this job.

- **Turnaround Time (TAT):** Arrival to Completion for every input j
- **Waiting Time (w):** Arrival to start of J_{pre} (Queue delay).
- **Response Time:** Arrival to start of J_{pre} .
- **Tail Latency:** Log the $P95$ and $P99$ latencies. This proves KairosSNNI prevents slow-client "convoy effects."
- **Utility Metric (Ψ):** Calculate the global Ψ for every experimental run.
- **SLO Attainment Rate:** The percentage of jobs that finished before their deadline.
- **Aggregate Tardiness:** Total milliseconds of delay for all late jobs (this captures the "Social Cost").
- **Goodput vs. Throughput:** Number of *on-time* results per minute vs. total results per minute.
- **Temporal Memory Occupancy:** Log the total RAM usage every 100ms
 - *Sub-log:* RAM used by **active jobs** vs. RAM used by the **proactive buffer**.
- **State-Dwell Time:** The time duration between the end of J_{pre} and the start of J_{on} for every request.
- **Buffer Hit Rate:** How many requests found a file ready in the 1TB project space vs. how many had to wait for the replenisher.
- **Core Utilization:** The percentage of the P cores used over time.

Asymmetric Client Performance (The "Asymmetry" Story)

These logs prove your Telemetry Engine and Negotiation are working.

- **θ_c Deviation:** Log (Estimated θ_c at Handshake) vs. (Actual θ_c measured at ReLU).
- **Negotiation Outcomes:** Log how many clients:
 1. Accepted n_{alt} (e.g., switched AlexNet to M2).
 2. Refused and took the **Hostage Core Penalty** ($\phi = 1$).
- **Hostage Core Duration:** Total time cores spent waiting for synchronous client responses.

9. Delivery Requirements

- **Modular C++ Code:** Logic must be separated into Config, FileManager, ResourceManager, and Scheduler.
- **Test Suite:** A script to simulate a Poisson arrival of 100+ requests with a configurable mix of job types.
- **Documentation:** Instructions on how to update profile.cfg for new models.

10. Code Reproducibility and Documentation

10.1 Mandatory Reproducibility:

The primary deliverable is a **Fully Reproducible Research Artifact**. The code must be architected

such that a third party can execute the entire pipeline—from request ingress to result logging—and achieve the same performance metrics reported by the Developer.

10.2 README Requirements:

The Developer must provide a comprehensive README.md file at the root of the repository. This file must include:

- **Environment Setup:** Detailed instructions for dependencies, SNNI protocol backend installation (SHARK/FSS), and hardware-specific configurations (Core Pinning/Project Space).
- **Execution Guide:** Step-by-step commands to run the Admission Control, the Schedulers, and the Dispatcher.
- **Experimentation Scripts:** Instructions on how to trigger the "Bursty Poisson" workloads and "Weak Client" scenarios.
- **Visualization:** Documentation on how to generate the results/graphs from the generated logs.
- The core goal here is **Academic Reproducibility**. This is a high-stakes research project. If I cannot run your code 6 months from now and get the exact same Ψ results, the project is considered incomplete. All IP stays with us, and the README must be 'dummy-proof'—meaning I should be able to follow it step-by-step to recreate your entire experiment."